

Database System Engineering and Distributed Backend Development

Course Code: 25CS1302E

Week-1

Week 1:

A local library is currently managing its books and members using a physical notebook, which has resulted in several operational problems such as misplaced books, difficulty tracking borrowed items, and inaccurate member records. To modernize its operations, the library plans to develop a database system called **BookFlow**. As a database developer, your task is to create the SQL foundation for this system. First, create a new database named **bookflow_db**. Then, design two tables: a **Books** table containing the columns **book_id**, **title**, **isbn**, and **published_year**, and a **Members** table containing the columns **member_id**, **full_name**, and **email**. Ensure data integrity by setting **book_id** and **member_id** as **PRIMARY KEYS**, applying a **NOT NULL** constraint to the **title** column so that every book must have a name, enforcing **UNIQUE** constraints on both **isbn** and **email** to prevent duplicate entries, and adding a **CHECK** constraint to ensure that the **published_year** is not in the future (for example, it must be less than 2027). Finally, insert three sample records into both the **Books** and **Members** tables to verify that the database structure and constraints are functioning correctly. This exercise will help you develop skills in schema definition, data integrity enforcement, and the use of SQL Data Definition Language (DDL) commands such as **CREATE DATABASE**, **CREATE TABLE**, and **INSERT** statements.

Solutions:

1. The Scenario

A local library currently tracks its books and members in a physical notebook. Books are being lost, and there is no way to know who has which book. The goal is to build the SQL foundation for "BookFlow" so that every book has a unique ISBN and no member can sign up without a valid (unique) email.

The requirements are:

- Create a database named **bookflow_db**.
- Create a **books** table (**book_id**, **title**, **isbn** **UNIQUE**, **published_year**) and a **members** table (**member_id**, **full_name**, **email** **UNIQUE**).
- Enforce **PRIMARY KEY** on both IDs, **NOT NULL** on title, and a **CHECK** constraint so **published_year** is not in the future (< 2027).
- Seed 3 books and 3 members, then test that the rules actually reject bad data.

2. Step 1 Database Setup

Query 1: Create the database

Explanation:

CREATE DATABASE creates a new, empty database (a container for tables). The IF NOT EXISTS clause makes the command safe to re-run: if bookflow_db already exists, MySQL skips creation instead of throwing an error. This is a DDL (Data Definition Language) command.

Output:

```
mysql> create database bookflow_db;  
Query OK, 1 row affected (0.01 sec)
```

Query 2: Select the database**Explanation:**

USE switches the current session to bookflow_db so that every subsequent CREATE TABLE, INSERT, and SELECT runs inside this database. Without this step, the tables would be created in whatever database was previously active.

Output:

```
mysql> use bookflow_db;  
Database changed
```

3. Step 2 Table Creation (DDL) with Constraints

Query 3: Create the books table**Explanation:**

Line-by-line breakdown of every column and constraint:

- book_id INT AUTO_INCREMENT PRIMARY KEY — the PRIMARY KEY uniquely identifies each row; it is automatically NOT NULL and UNIQUE. AUTO_INCREMENT makes MySQL generate 1, 2, 3, ... automatically, so we never insert the ID manually.
- title VARCHAR(255) NOT NULL — NOT NULL means a book cannot be saved without a name. An INSERT with a NULL title is rejected.
- isbn VARCHAR(13) NOT NULL UNIQUE — UNIQUE guarantees no two books share the same ISBN. VARCHAR is used instead of INT because ISBNs can have leading zeros and ISBN-10 can end in the letter X.
- published_year INT with CHECK (published_year < 2027) — the named CHECK constraint (chk_published_year) rejects any year of 2027 or later, preventing "future" books from entering the system.

```
mysql> CREATE TABLE books (  
->     book_id INT AUTO_INCREMENT PRIMARY KEY,  
->     title VARCHAR(255) NOT NULL,  
->     isbn VARCHAR(13) NOT NULL UNIQUE,  
->     published_year INT,  
->     CHECK (published_year < 2027)  
-> );  
Query OK, 0 rows affected (0.13 sec)
```

Query 4: Create the members table

Explanation:

- member_id INT AUTO_INCREMENT PRIMARY KEY — unique identity for every member, generated automatically.
- full_name VARCHAR(100) NOT NULL — a member must have a name on record.
- email VARCHAR(150) NOT NULL UNIQUE — NOT NULL means no signup without an email; UNIQUE means one account per email address. Together these satisfy the "no member can sign up without a valid email" rule.

Output:

```
mysql> CREATE TABLE members (  
->     member_id INT AUTO_INCREMENT PRIMARY KEY,  
->     full_name VARCHAR(100) NOT NULL,  
->     email VARCHAR(150) NOT NULL UNIQUE  
-> );  
Query OK, 0 rows affected (0.07 sec)
```


Query 5 (verification): Inspect the table structures

Output:

```
mysql> describe books;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| book_id        | int           | NO   | PRI | NULL    | auto_increment |
| title          | varchar(255)  | NO   |     | NULL    |                |
| isbn           | varchar(13)   | NO   | UNI | NULL    |                |
| published_year | int           | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.08 sec)

mysql> describe members;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| member_id      | int           | NO   | PRI | NULL    | auto_increment |
| full_name      | varchar(100)  | NO   |     | NULL    |                |
| email          | varchar(150)  | NO   | UNI | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

Explanation:

DESCRIBE shows the schema. Key = PRI confirms the PRIMARY KEY; Key = UNI confirms the UNIQUE constraint on isbn and email; Null = NO confirms NOT NULL columns. This is how an examiner verifies the constraints exist.

Step 3 Initial Seeding (Valid Data)

Query 6: Insert 3 books

Explanation:

A multi-row INSERT adds three books in a single statement. `book_id` is deliberately omitted — `AUTO_INCREMENT` assigns 1, 2, 3. All three rows pass every constraint: titles are present, ISBNs are unique, and all years are below 2027.

Output:

```
mysql> insert into books(title,isbn,published_year)values('The Alchemist', '9780061122415', 1988), ('Clean Code', '9780132350884', 2008), ('Atomic Habits', '9780735211292', 2018);
Query OK, 3 rows affected (0.06 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

Query 7: Insert 3 members**Explanation:**

Three members are registered, each with a distinct email, so the `UNIQUE` constraint is satisfied. `member_id` values 1–3 are generated automatically.

Output:

```
mysql> insert into members(full_name, email) values ('Anil Kumar', 'anil.kumar@example.com'), ('Priya Sharma', 'priya.sharma@example.com'), ('Ravi Verma', 'ravi.verma@example.com');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

SELECT * retrieves every column and row. Both tables show exactly 3 rows with auto-generated IDs, confirming the seeding succeeded.

5. Step 4 Constraint Tests (These MUST Fail)

Each query below deliberately violates one rule. A correct schema rejects all four. In an exam, showing the error message is the proof that data integrity works.

Test 1: Duplicate ISBN (UNIQUE violation)

```
INSERT INTO books (title, isbn, published_year)
VALUES ('Fake Copy', '9780061122415', 2000);
```

Output:

```
mysql> insert into books (title, isbn, published_year) values('Fake Copy', '9780061122415', 2000);
ERROR 1062 (23000): Duplicate entry '9780061122415' for key 'books.isbn'
```

Explanation:

The ISBN 9780061122415 already belongs to 'The Alchemist'. The UNIQUE constraint on isbn rejects the row, so two physical records can never claim the same ISBN.

Test 2: NULL title (NOT NULL violation)

```
INSERT INTO books (title, isbn, published_year)
VALUES (NULL, '9999999999999', 2010);
```

Output:

```
ERROR 1062 (23000): Duplicate entry '9780061122415' for key 'books.isbn'
mysql> insert into books (title, isbn, published_year) values('NULL', '9999999999999', 2010);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO books (title, isbn, published_year)
```

Explanation:

The NOT NULL constraint fires because a book must have a name. The database refuses to store a nameless book, exactly as the requirement demands.

Test 3: Future publication year (CHECK violation)

```
INSERT INTO books (title, isbn, published_year)
VALUES ('Time Traveler', '8888888888888', 2030);
```

Output:

```
mysql> INSERT INTO books (title, isbn, published_year)
      -> values('Time Travaler', '8888888888888',2030);
ERROR 3819 (HY000): Check constraint 'books_chk_1' is violated.
```

Explanation:

2030 fails the condition `published_year < 2027`, so the named CHECK constraint `chk_published_year` blocks the insert. Garbage dates cannot enter the table.

Test 4: Duplicate email (UNIQUE violation)

```
INSERT INTO members (full_name, email)
VALUES ('Anil Clone', 'anil.kumar@example.com');
```

Output:

```
mysql> insert into members (full_name, email) values ('Anil Clone', 'anil.kumar@example.com');
ERROR 1062 (23000): Duplicate entry 'anil.kumar@example.com' for key 'members.email'
mysql> |
```

Explanation:

`anil.kumar@example.com` is already registered to `member_id 1`, so the UNIQUE constraint on email rejects the second signup. One email = one account.

